

Security Audit Skill — Design

Artifact: `security-audit/` — a portable Claude Code skill that performs a standards-aligned security code audit of a given source path and emits a self-contained HTML report.

Relationship to the Modernizer. This skill is the *engine* specified by `Security-Module-Design-Plan.md`, extracted so it can run anywhere. The design plan describes a Modernizer-embedded module with a UI tab and a post-Phase-C hook; this skill implements the analysis, scoring, remediation-planning and reporting core of that module as a stand-alone, console-invoked component that the Modernizer can later call. UI integration is deliberately out of scope here.

1. What changed from the design plan, and why

Design plan	This skill	Reason
Launched from the Modernizer UI	Console-invoked (<code>/security-audit <path></code>)	Requested scope. A console entry point is also what CI needs, and the UI can shell out to it later.
Output <code>.md</code> + <code>.pdf</code>	Single self-contained <code>.html</code>	Requested. HTML gives filtering, search and a working checklist that neither format offers; the browser's own Print-to-PDF covers the PDF need, and the stylesheet has a print block for it.
.NET/Blazor-specific checks	Eight stack-keyed rule packs, plus a legacy pack	The skill must work in "any project", not only Modernizer output.
Deterministic scanners as code	Rule packs as data, executed through the built-in <code>Grep</code> tool	Removes the dependency problem entirely — see §3.
<code>BuildProbeService</code> / <code>AppRunProbeService</code> verification	Per-finding written verification steps	Those services belong to the Modernizer. Outside it, the skill states the check; it does not run the app.
—	Cost accounting in the report	Requested; also makes the "cost-aware confirmation" the plan called for possible, because there is now real per-run data to base an estimate on.

Everything the plan specified that survives unchanged: the OWASP taxonomy, the weighted two-score model (current + projected), the Discovered/Remediated checklist, the non-breaking remediation principle, and the phased roadmap. The `modernizer-v1` scoring profile reproduces the plan's exact category weights so reports stay comparable with `Informix-demo-3-Security-Analysis.md`.

2. Architecture

```
/security-audit <target-path> [options]
|
▼
Step 0 Cost watermark ..... bin/collect-cost.sh --mark
Step 1 Inventory & stack ..... Glob/Grep, confined to ROOT
Step 2 Deterministic pass ..... rules/*.json executed via the Grep tool
|                                     → candidates (hits) + absence checks
Step 3 Triage ..... references/triage-protocol.md
|                                     → read code, reachability, confidence, dedupe,
|                                     plus design findings grep cannot see
Step 4 Dependencies ..... manifests (+ optional OSV / advisory MCP)
Step 5 Score ..... references/scoring.md
Step 6 Remediation plan ..... references/remediation-playbook.md
Step 7 Cost ..... bin/collect-cost.sh --report
Step 8 Render ..... part1.html + findings.json + part2.html
▼
<Project> - Security analysis report. - YYYY-MM-DD.html
```

The layered design, and what each layer is for

Layer 1 — deterministic sensor (rule packs). 109 validated ripgrep patterns across eight packs. Fast, cheap, reproducible, and it never misses a pattern out of inattention. It is explicitly *not* a verdict layer: its output is called "candidates" throughout.

Layer 2 — model triage. Where the value is. The model reads the surrounding code, establishes whether untrusted input can actually reach the sink, looks for the mitigating control, assigns confidence honestly, deduplicates to root causes, and adds the design-level findings — missing authorization, IDOR, tenant isolation, business logic — that pattern matching structurally cannot see. `triage-protocol.md` makes this a procedure rather than a vibe.

Layer 3 — deterministic rendering. The model authors `findings.json` and never writes HTML. The template renders it. This keeps output consistent across runs, removes an entire class of escaping bugs, and spends tokens on analysis rather than markup.

That split is the central design decision. Each layer does what it is actually good at.

Absence checks

Several rules carry `"expect": "present-required"`, where a **zero-hit** result is the candidate finding — no `[Authorize]` anywhere in a web app, no security headers, no validation annotations. Pattern matching finds what is there; these find what is missing, which in access control is usually the whole finding. Each such rule carries a `precondition` so it cannot fire nonsensically (demanding `[Authorize]` in a console app).

3. Zero-dependency portability

Requirement: "as much as possible the skill will try to not require additional packages, so all the required ones should be portable with the skill."

The design target became *nothing to install at all*. The verification machine for this build had **no Node.js, and no reachable Python** — `python` on PATH was a 0-byte Microsoft Store alias stub, and a real CPython 3.12.10 was installed but invisible to the running process (see the Python section below). Only PowerShell 5.1, `dotnet` and Git Bash could be relied on. That is a very ordinary developer machine, and it is exactly the case a Python-first design would have failed on — which is why the audit path depends on none of them:

Need	Conventional approach	What this skill does
Scan the code	A Python/Node scanner + regex library	Rule packs as JSON data, executed by the built-in <code>Grep</code> tool (ripgrep, ships with Claude Code)
Render the report	A templating library	Two static HTML halves concatenated around the JSON with <code>cat</code>
Account for cost	A JSON-parsing script	Three interchangeable collectors — see below

Total runtime dependencies: **none that need installing**. Cost accounting has three implementations and uses whichever is present:

Collector	Requires	Available on
<code>bin/cost.py</code>	Python 3.9+, stdlib only	Preferred when a Python already exists
<code>bin/collect-cost.sh</code>	bash + awk	Git Bash on Windows, native macOS/Linux
<code>bin/collect-cost.ps1</code>	PowerShell 5.1	Every Windows 10/11 machine

All three produce byte-identical `cost.json`; that equivalence is verified, not assumed. If none can run, the audit still completes and the report marks cost `unavailable`.

Python: optional accelerator, never a prerequisite

Python is used when it happens to be there and ignored when it isn't. `bin/ensure_python.*` resolves an interpreter through five steps: `$SECURITY_AUDIT_PYTHON` → PATH → well-known install locations and the registry PATH → a cached bootstrap → a pinned download that requires explicit consent.

It probes by executing code rather than trusting PATH, and the verification machine proved why twice over:

- `%LOCALAPPDATA%\Microsoft\WindowsApps\python.exe` is a **0-byte App Execution Alias** that resolves on PATH, prints a Store advert and exits non-zero. Anything using `command -v` picks it and breaks.
- The machine's registry PATH was *correct* — Python312 ahead of WindowsApps — but the running Claude Code process had started before Python was installed, so its in-memory PATH had no Python at all. Step 3 found the interpreter regardless. A PATH-trusting resolver would have failed on a correctly configured machine.

The last-resort bootstrap (`assets/python-bootstrap.json`) pins an exact CPython version and its SHA-256 for six platforms, verifies the archive **before** extraction, fails closed on mismatch, and installs into the user cache — never the skill folder, never the audited repo. It is never automatic.

That strictness is a consistency requirement, not ceremony: the skill's own `CFG-CI-02` rule flags curl-piped-to-shell and unpinned dependencies as findings. A security tool that did the thing it warns about would forfeit the credibility that is its only asset.

The concatenation trick is what makes the report dependency-free. `report.part1.html` ends with an open `<script type="application/json">` tag; `report.part2.html` begins with the matching close tag and contains the renderer. `cat part1 findings.json part2 > report.html` produces a single valid self-contained file with no build step and no library.

4. Scope confinement

The skill takes a target path and treats it as a hard boundary:

- ROOT is resolved to an absolute path; every analysis `Grep`, `GLob` and `Read` is rooted there.
- Upward traversal is prohibited outright.
- Reads outside ROOT are limited to the skill's own files, the output directory, and the session transcript (for cost only — its contents never enter the report).
- Every path in the report is **relative to ROOT**, so reports are portable and do not leak the auditor's local directory layout.

This matters for correctness as much as for safety: an audit that wanders into a sibling project produces findings the reader cannot act on and a score that describes nothing.

5. Standards alignment

Standard	Role	Handling
OWASP Top 10 (2021)	Primary axis; drives category weights	Every finding carries a category
OWASP Top 10 (2025)	Secondary	Optional <code>owaspCurrent</code> , populated only after fetching the live list — never from recall
CWE	Precise defect identifier	45-entry curated table in <code>standards-mapping.md</code>
MITRE ATT&CK	Adversary behaviour, for the security team	~20-technique table; indicative mapping only
NIST SP 800-218 (SSDF)	Process gaps, for auditors	Practice ids (PW.5, PW.9, RV.1 ...)
NIST SP 800-53 Rev. 5	Control families, for GRC	AC-3, IA-2, SC-28, RA-5 ...

The no-invented-identifiers rule is enforced throughout. `SKILL.md`, `triage-protocol.md` and `standards-mapping.md` all state it, and the schema marks every identifier field optional so omission is always available. A fabricated CVE or a wrong CWE discredits every other number in the document, and the reader has no way to tell which ones are wrong.

On the 2021-versus-2025 choice

2021 is the primary axis because it is what tracking systems, compliance mappings and developer tooling are keyed to today. The skill can report the current edition alongside it, but only by fetching the live list — precisely the kind of detail that should not come from model recall.

6. Scoring

Weighted category scores → single 0–100 score, letter grade, and posture band. Full arithmetic in `references/scoring.md`, reproduced in the report's methodology so a reader can recompute it.

Three deliberate design choices:

Confidence multiplies the penalty (Confirmed 1.0, Likely 0.75, Possible 0.4). Uncertain findings still appear but move the number less, which removes the incentive to overstate.

Unassessed categories score `null`, not 100. Their weight is redistributed and they are listed in Limitations. Scoring an unchecked category as perfect is the single easiest way to turn a security report into a rubber stamp.

Posture bands override the score. Any single Critical forces "At Risk" regardless of the number. An application with one unauthenticated admin endpoint is not "Hardened" at 91/100.

Two profiles ship: `default-v1` (11 categories, full OWASP coverage) and `modernizer-v1` (the design plan's exact 8 weights, for continuity with existing Modernizer reports).

7. Cost accounting

Requirement: the report states the analysis cost in tokens and dollars.

Measured, not estimated, wherever possible:

1. **Watermark.** Before any analysis, record the session transcript path and its current line count.
2. **Extract.** After the audit, the collector replays only the lines added since the watermark. Three interchangeable implementations exist — `cost.py`, `collect-cost.sh` (via `cost.awk`) and `collect-cost.ps1` — so this works whatever the machine has.
3. **Deduplicate.** A single assistant message appears in the transcript several times (streaming partials plus a final record) — the verification transcript held 5,214 assistant lines for 2,389 distinct messages. Naive summing overstates cost by roughly 2×. The extractor keys on message id and keeps the maximum value seen per field.
4. **Price per TTL.** The transcript distinguishes `ephemeral_5m_input_tokens` from `ephemeral_1h_input_tokens`, and those bill at different multiples of the input rate (1.25× and 2×). Collapsing them would misprice a cache-heavy run substantially.
5. **Report honestly.** `source` is `transcript`, `estimate`, or `unavailable`, and the template renders a visible caveat for anything that is not measured. Unknown models report token counts with a null cost rather than a guessed rate.

Rates live in `assets/pricing.json` with a maintenance note: they are list prices as of a stated date, and they do not apply to Bedrock, Vertex, Batch API or enterprise agreements.

Verified end to end against a real 11,784-line transcript, including the watermark, the subagent-transcript sweep, and the `unavailable` fallback path. All three collectors were run over the same window and agree to the token: 36,514,698 tokens / \$37.2258.

That three-way check was not ceremony — it caught a bug nothing else would have. Git Bash records the watermark path as `/c/Users/...`; native Python cannot open that, silently fell back to scanning the entire transcript, and reported **\$892 instead of \$37**. A 32× overstatement that looked entirely plausible in isolation. `cost.py` now translates MSYS paths. The lesson generalizes: a fallback that degrades silently is worse than one that fails loudly, and only differential testing finds it.

8. The report

Single self-contained HTML file, named exactly `<Project name> - Security analysis report. - <YYYY-MM-DD>.html`.

Sections: masthead and scope · score dashboard (current / projected / uplift, severity KPI tiles) · category breakdown (grouped bars **and** a data table) · controls verified present · findings summary with Discovered/Remediated checkboxes · detailed findings · remediation roadmap · analysis cost · scan evidence · limitations.

Interactive: severity filters, full-text search, expand-all, a persisted remediation checklist, a "copy checklist as Markdown" button, and a theme toggle. All of it degrades to a readable static document with JavaScript disabled at print time.

Visualization follows the `dataviz` skill's method: form chosen before color (hero numbers for scores, not fake gauges), categorical slots 1–2 in fixed order for the two series, the reserved status palette for severity — always with an icon and a text label so color never carries meaning alone — 4px rounded data-ends, a 2px surface gap between adjacent bars, recessive grid, a legend for the two series, direct value labels, and a table view of the same data. The palette is the skill's documented default used verbatim; `node` was unavailable on the build machine, so it was not re-run through `validate_palette.js` — re-validation is required only when substituting different ramps.

Why HTML rather than Markdown + PDF

The plan specified `.md` + `.pdf`. HTML was chosen because the report's primary use is *working through findings*, not reading start to finish: filtering to Criticals, searching for a filename, ticking items off as they land. Markdown cannot do any of that, and the PDF requirement is met by the browser's own print path, for which the stylesheet includes a print block that expands every finding and suppresses the controls.

9. Verification performed on this build

Check	Method	Result
All rule regexes compile	Extracted with a real JSON parser, each fed to <code>rg</code>	109/109 pass. Two initial failures found and fixed: a backreference (<code>\1</code>) in the Spring CSRF rule and a negative lookahead in the cleartext-HTTP rule — neither is supported by Rust regex
Cost extractor correctness	Run against a real 11,784-line transcript	Dedupe verified (2,389 messages from 5,214 lines); per-TTL cache pricing correct; watermark scoping correct
Cost fallbacks	<code>--mark</code> with no prior state; <code>--report</code> with no transcript	Both produce valid JSON; the no-transcript path reports <code>source: unavailable</code>
Report JSON contract	Example findings parsed with a real JSON parser	Valid; 7 findings, 8 categories
Renderer syntax	Extracted and parsed by the Windows JScript engine	No syntax errors
Renderer logic	Executed under a DOM shim against the example data	Correct element counts; 3 bugs found and fixed — a double-escaped entity in the masthead, an HTML-unsafe <code><</code> in the currency formatter, and bar labels overflowing at high values
Visual output	Headless Edge screenshots, light and dark	Renders correctly in both; 2 layout bugs found and fixed — finding IDs wrapping in the summary table, and the finding title running into its metadata
Python resolution	Run with a stale PATH and a 0-byte alias stub in place	Both resolvers rejected the stub and located the real 3.12.10 via well-known locations
Three-way collector agreement	Same transcript window through awk, PowerShell and Python	Identical to the token: 36,514,698 tokens / \$37.2258 from all three. One interop bug found and fixed — Git Bash writes <code>/c/Users/...</code> watermarks that native Python cannot open, and the silent fallback to a full-file scan inflated the reported cost ~32×
Rule-pack corpus	<code>tests/run_corpus.py</code> , 109 patterns + 14 fixtures	3 real rule defects found on the first run — see below

The shipped sample report (`security-audit/examples/`) is the output of that verification run.

What the corpus runner caught immediately

Its first execution found three defects that had survived manual review:

1. **UNI-INJ-01 was exactly backwards.** Its `[^'"\\n]` gap excluded quote characters, so it could not match concatenated SQL (which necessarily crosses a quote) while it *did* match C# `$("#...{x}...")` interpolation — which `FromSqlInterpolated` parameterizes safely. It missed the vulnerability and flagged the safe form. Rewritten to key on the concatenation operator, and the bare-brace alternative dropped: the language packs judge interpolation with the context needed to tell parameterized from raw.
2. **A false positive on `"your-api-key-here"`.** Resolved honestly rather than by patching the regex — a pattern cannot separate a wordy placeholder from a real credential without a wordlist, and a wordlist

would be incomplete while creating false confidence. The fixture now documents the division of labour: the sensor fires, triage drops it.

- 3. The pattern-compile check was folded in, so the two Rust-regex failures found earlier can never silently return.

This is the argument for the corpus in one paragraph: all three were invisible to inspection, and two of them made the tool *wrong in the dangerous direction* — silent on a vulnerability, noisy on safe code.

10. Deliberate non-goals

- **The skill does not modify the code it audits.** Applying fixes is a separate, explicitly requested action. An auditor that edits is an auditor whose findings cannot be trusted.
- **No runtime testing.** Static analysis only; stated plainly in every report's Limitations.
- **No bundled MCP server.** The core audit needs no network. Dependency-advisory confirmation is the one place network access genuinely helps, and the skill advertises loudly when it lacked it rather than implying the dependencies are clean.
- **No secret-scanning history rewrite.** The skill reports committed secrets and says rotation is the remediation; it does not touch git history.

11. Modernizer integration path

The skill is the analysis core the design plan asked for. To embed it:

Plan element	How it maps
SecurityAnalysisService	Shell out to <code>claude -p "/security-audit <generatedAppPath> --name <app>"</code> after Phase C
SecurityReviewAgent	The skill itself — <code>SKILL.md</code> is the system prompt and procedure
RemediationPlanner	<code>--mode plan</code> , driven by <code>remediation-playbook.md</code>
Plan-A SecurityReviewMode combo	Maps to <code>--mode</code> and <code>--depth</code>
Cost-aware confirmation dialog	Read the previous run's <code>cost.json</code> ; the depth tiers give a ~1 : 3 : 8 relative estimate
Security tab	Read <code>findings.json</code> for the table and scores; link the HTML for the full report
Before/after comparison	Re-run with <code>--baseline <previous findings.json></code> ; findings come back tagged New / Persisting / Resolved
Build + smoke verification	Modernizer-side: run <code>BuildProbeService</code> and <code>AppRunProbeService</code> against each finding's <code>verification</code> text

The one thing the Modernizer should add that this skill deliberately omits: **auditing the legacy source as well as the generated output**. With both, `origin` classification becomes evidence rather than inference, and the `migration-introduced` category — flaws the generator created, which are fixable once in the generator rather than per project — becomes reliably identifiable.